5. Write the python program for Missionaries Cannibal problem

Aim

To develop a Python program to solve the Missionaries and Cannibals Problem using the Breadth-First Search (BFS) Algorithm by finding a safe sequence of moves to transfer all missionaries and cannibals across the river.

Algorithm

1. Start the program.
2. Initialize the starting state as (3 Missionaries, 3 Cannibals, Boat on Left Bank).
3. Define the goal state as (0 Missionaries, 0 Cannibals, Boat on Right Bank).
4. Insert the initial state into a queue.
5. Repeat until the queue becomes empty:
    - Remove the front state.
    - If the current state is the goal state, display the solution path and stop.
    - Generate all valid boat moves.
    - Check whether each generated state satisfies the safety conditions.
    - Add all valid and unvisited states to the queue.
6. If no solution exists, display "No Solution Exists."
7. Stop.

Program

```
from collections import deque
# Check whether a state is valid
def is_valid(state):
    M_left, C_left, boat = state
    M_right = 3 - M_left
    C_right = 3 - C_left
    # Check limits
    if M_left < 0 or M_left > 3 or C_left < 0 or C_left > 3:
        return False
    # Missionaries should not be outnumbered on left bank
    if M_left > 0 and C_left > M_left:
```

```
        return False
    # Missionaries should not be outnumbered on right bank
    if M_right > 0 and C_right > M_right:
        return False
    return True
# Generate possible next states
def get_next_states(state):
    M_left, C_left, boat = state
    moves = [
        (1, 0),   # 1 Missionary
        (2, 0),   # 2 Missionaries
        (0, 1),   # 1 Cannibal
        (0, 2),   # 2 Cannibals
        (1, 1)    # 1 Missionary and 1 Cannibal
    ]
    next_states = []
    for m, c in moves:
        if boat == 0:  # Boat on left side
            new_state = (M_left - m, C_left - c, 1)
        else:          # Boat on right side
            new_state = (M_left + m, C_left + c, 0)
        if is_valid(new_state):
            next_states.append(new_state)
    return next_states
# BFS Algorithm
def bfs():
    start = (3, 3, 0)   # (Missionaries Left, Cannibals Left, Boat)
    goal = (0, 0, 1)
    queue = deque([(start, [])])
    visited = set()
    while queue:
        state, path = queue.popleft()
        if state in visited:
            continue
```

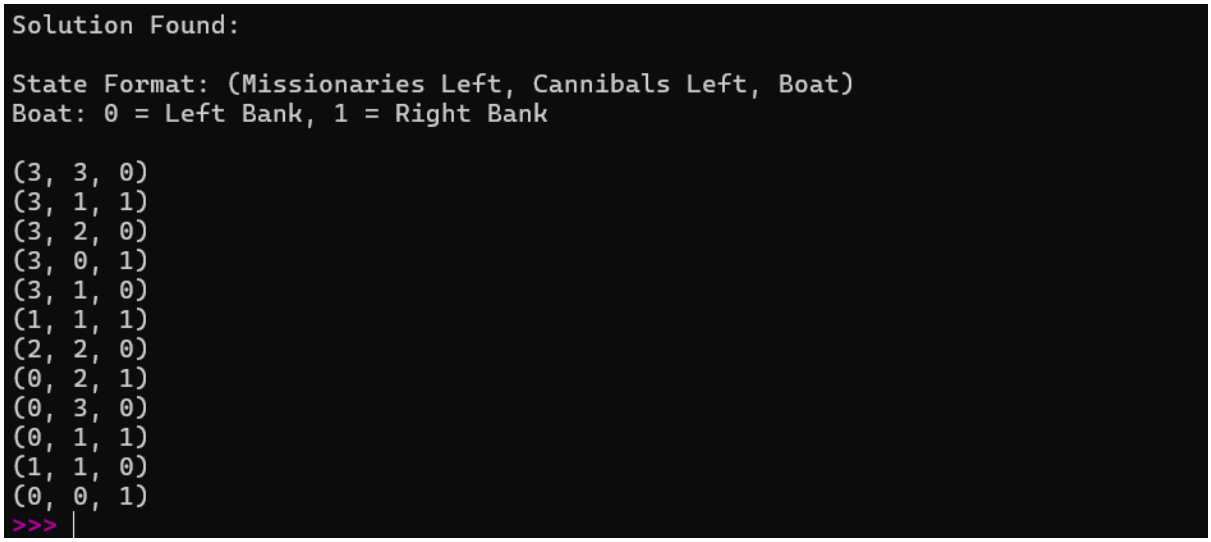
```
        visited.add(state)
        path = path + [state]
        if state == goal:
            return path
        for next_state in get_next_states(state):
            if next_state not in visited:
                queue.append((next_state, path))
    return None

# Main Program
solution = bfs()
if solution:
    print("Solution Found:\n")
    print("State Format: (Missionaries Left, Cannibals Left, Boat)")
    print("Boat: 0 = Left Bank, 1 = Right Bank\n")
    for step in solution:
        print(step)
else:
    print("No Solution Exists.")
```

Input

No user input required.

(The problem starts with 3 Missionaries, 3 Cannibals, and the boat on the left bank.)

Output



Result

The Python program was successfully implemented to solve the Missionaries and Cannibals Problem using the Breadth-First Search (BFS) Algorithm. The program finds the shortest valid sequence of moves that transfers all missionaries and cannibals safely from the left bank to the right bank without violating the safety constraints, and displays each intermediate state of the solution.